

C22 — PM-Level System Design

1. What interviewers test

"Design a service that helps people find lost devices without exposing their location to others." "Design the notifications system for a social app." "How would you build 'where is my order'?" The system design question asks you to design a product-and-technology system out loud. For a PM it is not a test of distributed-systems knowledge. It tests whether you can turn a vague product request into requirements, estimate scale well enough to choose between designs, sketch the parts and the flow of data, state the tradeoffs among speed, freshness, cost and privacy, anticipate what breaks and who might abuse it, and explain the result to engineers and to executives. Candidates fail it in two opposite ways: by diving into boxes and databases before knowing what the product must do, and by staying so abstract that nothing could be built.

Where it shows up. Inferred from the round structures candidates commonly report: system design or technical-design questions for PM roles at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, more often for technical-leaning roles and in later rounds, usually 30 to 45 minutes [unverified: candidate-reported]. For associate roles the bar is a coherent, sensible design with good questions, not engineering depth. On the job it is the work of design reviews: framing the problem so engineers can design well, and checking that the design serves users.

What is scored. Whether your design is driven by the user and the constraints Inferred. Interviewers listen for requirements asked before components are drawn; a scale estimate that sizes the problem; a clear picture of the main data flow; named tradeoffs with a reasoned choice; failure modes with what the user sees; abuse cases where the system touches people or money; and an explanation that a non-engineer could follow.

Six observable behaviors:

1. You clarify who the users are, what the system must do, the scale and the constraints before drawing anything.
2. You estimate load and storage with a stated method, and use the result to decide.
3. You sketch the components and the flow of a read and a write, at a level a PM can defend.
4. You name the tradeoffs (latency, freshness, cost, privacy) and choose with a reason.
5. You describe failure modes, degraded behavior and abuse cases, and what the user sees.
6. You summarize the design for engineers and for an executive, and say what you would validate first.

The failure this prevents. Six patterns end system design answers: starting with technology, no numbers, boxes with no data flow, "it's scalable" with no reasoning, no thought of failure or abuse, and an answer that engineers could not act on or executives could not follow.

2. The concept from first principles

C22.1 What the question tests, and what it does not

A PM-level design answer is a product decision expressed as a system. It sits between two other kinds of question. *Product design* (C07) asks what to build for whom. *Technical fluency* (C21) asks whether you understand how systems work. System design joins them: given what to build, how would the pieces fit, what does it cost, and what could go wrong? You are not expected to choose a database engine or design a sharding scheme; you are expected to know that such choices exist and to ask engineering the right questions about them.

What to do and what not to do. Do ask clarifying questions, state assumptions, draw a simple diagram, use numbers, name tradeoffs and failure modes, and say what you would ask an engineer. Do not recite technologies to sound expert, claim certainty about details you do not know, or build a design that ignores the user. "I would ask the infrastructure team whether a relational store handles this write volume, because I estimate about 9,000 writes a second at peak" is a better answer than naming a product.

When not to use a full design. If the interviewer asks a narrow question (how would you speed up this page?), use C21's tools. Use the full structure when the prompt is "design X" and the system has users, data, scale and failure consequences.

Counterexample. Asked to design a notifications system, a candidate says: "I'd use a message queue, a microservice for each channel, and a NoSQL database, with Kubernetes for scaling." The interviewer asks how many notifications a day and what happens at 9 a.m. when everyone's digest is due. The candidate has not asked who the users are or estimated the load, and cannot say what the technology choices are for. A stronger start: "Before drawing anything: who sends and who receives, roughly how many a day, how fast must they arrive, and what must never happen, such as sending someone the same notification ten times?"

Hidden risks. Two. *Technology theater*: naming tools without reasons reads as memorization. *Skipping the user*: a design that is technically neat and ignores privacy, abuse or the user's wait fails the product test.

C22.2 Answer architecture for 45 minutes

FW-C22-01 sets out the spine. A 45-minute round divides roughly as follows:

Step	Minutes	What to produce
Clarify the problem	5	Users, the core job, what is in and out of scope
Requirements	7	Functional and non-functional requirements, constraints, success measures
Scale estimate	5	Users, requests per second at average and peak, storage, one cost figure
High-level design	10	Components and the data flow for the main read and write paths
Deep dive	8	One or two parts that matter most, with tradeoffs
Failure, abuse and degradation	6	What breaks, what the user sees, who might misuse it
Measures, rollout and summary	4	How you would know it works, a staged launch, a closing summary

For 30 minutes shorten the clarifying and estimation steps and keep one deep dive. Say the plan aloud at the start ("I'll clarify, estimate, sketch, go deep on one part, then cover failure and abuse") so the interviewer can steer.

C22.3 Requirements first: users, scale, constraints

Requirements are what the design must satisfy, and the questions you ask to find them are part of the answer (FW-C22-02).

Users and the core job. Who uses it (consumers, businesses, other systems)? What is the main thing they want to do, in one sentence? Who else is affected (recipients, bystanders, regulators)?

Functional requirements. What must the system do? Keep the list short and ordered: for a notification system, send a message to a user on one or more channels, respect the user's preferences, and avoid duplicates. Name what is out of scope for now (a non-goal, C16.2), such as an editor for message templates.

Non-functional requirements. How well must it do it? *Latency* (how fast must a notification arrive: seconds, minutes?), *freshness* (how current must the data be?), *availability* (what must not go down, C21.7), *durability* (what must never be lost, such as a payment record), *consistency* (must everyone see the same thing at once?), *cost* (a budget per user or per request), and *privacy and security* (what must stay secret, who must not see what).

Scale. How many users, how many actions per user per day, how much data, how bursty? You may be given numbers; if not, state assumptions and ask whether they are reasonable.

Constraints. Regulation (data location, consent), devices and networks (C21.9), existing systems, team size and time, and what the system depends on.

Success measures. What would tell you it works? A primary metric tied to the user's job, with guardrails (C05).

Why it works. Requirements turn an open prompt into a bounded problem and show judgment. **When not to ask everything.** Limit yourself to the five or six questions whose answers change the design, and state assumptions for the rest.

C22.4 Components and data flow

A PM-level sketch has a handful of boxes and arrows (FW-C22-04). The standard parts, from C21, are:

- **Clients** (apps and browsers), which send requests and show results.
- **An entry point** (a load balancer or gateway) that receives requests, checks who the caller is, and limits abusive traffic (**rate limiting** is capping how many requests a caller may make in a period, to protect the system).
- **Application services** that hold the logic: one or a few, named by what they do ("notification service," "feed service").
- **Data stores:** a main database that is the system of record, perhaps a search index, a file or object store for photos and videos, and a **cache** for hot data.
- **A message queue** that decouples fast request handling from slower work (sending, resizing, ranking).
- **External services:** payment partners, push-notification providers, maps.
- **Background jobs and pipelines** for batch work and analytics (C21.4).

Draw the flow, not just the boxes. Trace the main *write* path (what happens when a user does something) and the main *read* path (what happens when someone looks), and number the steps. For a photo upload: the app sends the photo to the entry point; the service stores the file in object storage and a record in the database; it puts a "process this photo" message on a queue; a worker makes thumbnails; a notification goes to followers. Say what is synchronous (the user waits) and what is asynchronous (the user does not).

Choose storage by the question asked of it, at PM depth. Records you look up by identifier and must not lose (users, orders): a relational database. Large files: object storage. Fast lookups of recent items: a cache. Free-text or filtered search: a search index. You can say, "I would ask engineering to choose, but the access pattern is a lookup by user, with writes that must not be lost, which points to a database with strong guarantees."

C22.5 Capacity estimation

Estimation sizes the problem and drives choices (C04 gave the method; FW-C22-03 is the sheet). Use a small set of formulas.

- **Requests per second (average)** = daily users × actions per user per day ÷ 86,400.
- **Peak** = average × a peak factor. Traffic is bursty: consumer products often peak at three to ten times the average, and event-driven ones (a sale, a launch, a morning digest) more **Inferred**. State the factor and why.
- **Storage** = items per day × size per item × retention period (and any copies).
- **Bandwidth** = requests per second × size of the response.

- **Cache size** = the size of the hot set (the small share of items that draw most requests) times item size.
- **Cost** = requests $\times$ cost per request, or storage $\times$ price per month.

Example (invented). A messaging app has 10,000,000 daily users, each sending 20 messages a day: 200,000,000 messages a day, 2,315 a second on average; at a peak factor of 5, about 11,600 a second. At 1 KB a message, the daily storage is 200 GB; kept for a year (before any compression or copies), about 73 TB. Each of these numbers suggests an engineering question: can one database take 11,600 writes a second, or does it need to be split? Is 73 TB cheap enough to keep, or should old messages move to cheaper storage?

Use the numbers to choose. The point of the estimate is a decision: if peak writes are 100 a second, a single database is fine; if 100,000, the design must spread the load. If a response is 5 KB and there are 10,000 a second, bandwidth is 50 MB/s, trivial; at 5 MB (video) it is not. Say so aloud: "At about 9,000 writes a second I would not assume one database is enough; I'd ask engineering."

Sanity-check against what you know (C04's two-route check). A figure that implies more messages than there are people-hours in the day is wrong.

C22.6 Tradeoffs: latency, consistency, cost, privacy

Most design choices trade four things (FW-C22-05). Name which two you are trading and choose with a reason.

Latency against freshness. Serving from a cache is fast and may be stale; computing on demand is fresh and slow (C21.5). Decide by the user: a like count can be seconds old; an account balance cannot.

Consistency against availability. When copies of data exist in several places, the system can insist that everyone sees the same value (and refuse to answer when it cannot be sure) or answer with what it has (and allow brief disagreement). **Eventual consistency** means copies may differ for a short time but will agree if no new changes arrive. Choose strong consistency where a mistake is costly (money, inventory counts at checkout) and eventual consistency where a brief difference is harmless (view counts, feed order).

Cost against everything. Faster, fresher, more redundant and more private all cost more. Express it as cost per user per month and compare with revenue per user.

Privacy against capability. What you collect and where you process it limit what you can build and what can leak (C21.8). Collecting less, processing on the device and encrypting so that even the company cannot read the data are strong privacy designs with costs in features and in the ability to investigate abuse. Say what you give up.

Push against pull. *Push* sends updates to the user as they happen (fast, costs connections and battery); *pull* has the user's app ask periodically (simple, delays updates, wastes calls when nothing changed). Worked example 2 shows the same choice in a feed.

C22.7 Failure modes and graceful degradation

Graceful degradation is a system's ability to keep providing a reduced but useful service when a part fails, instead of failing completely. A product page that loads without recommendations when the recommendation service is down is gracefully degraded; a blank page is not. FW-C22-06 is the table: for each component, how it can fail, what the user sees, how the system degrades, and the signal that tells you.

Common failures and defenses. Overload (shed load, queue work, **rate limiting**), a slow dependency (time out and use a fallback), duplicates from retries (idempotency, C21.3), lost messages (queues with acknowledgment and retry, and a way to reconcile), a bad deploy (staged rollout and rollback, C17), a data store outage (read-only mode from a cache), and a region outage (redundancy, at a cost). Say what the user sees in each: a stale value labeled with its age, a "try again" with the work saved, a queued action marked "sending."

Abuse cases. An **abuse-case** is a way the system could be used to harm someone, analyzed like a requirement. Features that touch people, location, messages or money need an abuse pass (FW-C22-08): who might misuse it (a stalker, a scammer, a harasser, a bot), how, what harm would result, and what control reduces it (limits, friction, detection, user notification, easy removal). A location-finding service that works for owners works for stalkers unless the design prevents it, as Worked design in IQ-C22-06 shows.

Rehearse failure. Teams test failure on purpose (game days), and PMs should ask what has been tested and what the rollback is (C17.5). CS-C22-02 shows a system that was designed for a likely load and met an unlikely one.

C22.8 Worked designs: notifications, feed, search, payments flow, link service

Section 5 works five common designs at PM depth: a notifications system, a news feed (push against pull), search over listings, a payments flow with retries and webhooks (a **webhook** is a message one system sends to another's web address when an event happens, so the receiver need not keep asking), and a link-shortening service. In each, the steps of the spine appear: requirements, estimate, sketch, tradeoff, failure and abuse. Another pattern in several designs is **fan-out**: one action causing many deliveries or updates, such as one post reaching a million followers; it is the main cost driver in feeds and notifications (not to be confused with the join fan-out of C21.4, where rows repeat).

C22.9 Communicating the design to engineers and to executives

The design is useful only if others can act on it (FW-C22-07). A one-page summary has: the problem and the user in one sentence; the requirements that drive the design; the estimate (the two or three numbers that matter); the sketch with the main flows; the key tradeoffs and the choices; the failure and abuse cases with mitigations; the open questions for engineering; what you would build first and how you would roll it out and measure it.

For engineers: lead with assumptions, numbers and open questions, in precise terms, and leave the how to them. *For executives:* lead with the user outcome, the risk (what could go wrong and how bad), the cost in money and time, and the decision you need; use plain language and one diagram. In an interview, finish with a 60-second summary in the executive form, then invite the interviewer to probe the engineering detail.

3. Frameworks

Eight frameworks, set out below. The request-path map (FW-C21-01), the engineer conversation questions (FW-C21-02), the latency budget (FW-C21-03), the architecture tradeoff table (FW-C21-05), the reliability targets sheet (FW-C21-06), the estimation protocol (FW-C04-01), the anchor ladder (FW-C04-03) and the rollout ladder (FW-C17-02) are reused, not repeated.

FW-C22-01 · The PM system-design spine · Original

- **Inputs:** a design prompt and 30 to 45 minutes.
- **Outputs:** an answer with seven visible steps: clarify, requirements, estimate, high-level design, deep dive, failure and abuse, measures and summary.
- **Assumptions:** the interviewer will answer clarifying questions or accept stated assumptions.
- **Tradeoffs:** structure shows judgment and can crowd out depth on the one part that matters.
- **Failure modes:** starting with technology; no numbers; no failure or abuse pass.
- **Worked example:** the designs in section 5 and the model answers in section 11.

Steps: **Clarify → Requirements → Estimate → Design (read and write paths) → Deep dive → Failure and abuse → Measures, rollout, summary.**

FW-C22-02 · The requirements card · Original

- **Inputs:** a vague prompt.
- **Outputs:** a card with the users and core job, functional requirements (ordered), non-functional requirements (latency, freshness, availability, durability, consistency, cost, privacy), scale, constraints, non-goals and success measures.
- **Assumptions:** the five or six questions that change the design can be asked.
- **Tradeoffs:** more questions delay the design; stated assumptions save time.
- **Failure modes:** a feature list with no non-functional requirements; no non-goals.
- **Worked example:** the cards in Worked examples 1 and 5.

Item	Answer
Users and core job	
Must do	

Item	Answer
Must not do	
Latency and freshness	
Availability and durability	
Scale (users, actions, peak)	
Privacy and constraints	
Non-goals	
Success measure and guardrail	

FW-C22-03 · The capacity estimation sheet · Original

- **Inputs:** daily users, actions per user, item sizes, retention and a peak factor.
- **Outputs:** average and peak requests per second, daily and total storage, bandwidth, cache size and one cost figure, each with the engineering question it raises.
- **Assumptions:** inputs are stated and sanity-checked.
- **Tradeoffs:** rough numbers decide between designs; precision is not the aim.
- **Failure modes:** no peak factor; averages used for design; numbers with no decision attached.
- **Worked example:** all five in section 5.

Formulas: **average per second** = $\text{users} \times \text{actions} \div 86,400$; **peak** = $\text{average} \times \text{factor}$; **storage** = $\text{items} \times \text{size} \times \text{retention}$; **bandwidth** = $\text{per second} \times \text{response size}$.

FW-C22-04 · The component and data-flow sketch · Original

- **Inputs:** the requirements and the estimate.
- **Outputs:** a diagram with clients, an entry point, services, stores, a cache, a queue and external services; a numbered write path and read path; and what is synchronous and what is asynchronous.
- **Assumptions:** the standard parts fit; non-standard ones are named.
- **Tradeoffs:** a simple sketch can be defended; a detailed one invites questions you cannot answer.
- **Failure modes:** boxes with no flow; unlabeled arrows; no mark of where the user waits.
- **Worked example:** the flows in Worked examples 1 to 5.

FW-C22-05 · The tradeoff dials · Original

- **Inputs:** a design decision.
- **Outputs:** the two dials being traded (latency against freshness, consistency against availability, cost against quality, privacy against capability, push against pull), the choice and the reason tied to the user.
- **Assumptions:** the cost of being wrong is known for the data involved.

- **Tradeoffs:** every setting helps one thing and costs another.
- **Failure modes:** "both"; no reason; choosing by habit.
- **Worked example:** Worked example 2 and IQ-C22-08.

Data	Cost of stale or wrong	Choice
Like count	Low	Eventual; cache
Account balance	High	Strong consistency

FW-C22-06 · The failure-mode table · Original

- **Inputs:** the components in the sketch.
- **Outputs:** for each component: how it fails, what the user sees, the graceful degradation, the mitigation, and the signal.
- **Assumptions:** failures are expected, not exceptional.
- **Tradeoffs:** every mitigation costs; spend where harm is highest.
- **Failure modes:** failures listed with no user view; no signal.
- **Worked example:** Worked examples 1, 4 and CS-C22-02.

Component	Failure	User sees	Degradation	Signal
-----------	---------	-----------	-------------	--------

FW-C22-07 · The design one-pager · Original

- **Inputs:** the finished design.
- **Outputs:** a one-page summary with the problem, requirements that drive the design, the numbers, the sketch, tradeoffs, failure and abuse cases, open questions, and the first build and rollout; in an engineer version and an executive version.
- **Assumptions:** the audience is named.
- **Tradeoffs:** brevity loses detail; link to detail rather than include it.
- **Failure modes:** one document for both audiences; no open questions; no decision requested.
- **Worked example:** IQ-C22-10.

FW-C22-08 · The abuse-case pass · Original

- **Inputs:** a feature that touches people, location, messages or money.
- **Outputs:** a table of who might misuse it, how, the harm, and the control (a limit, friction, detection, notification to the affected person, easy removal), plus the residual risk and who accepts it.
- **Assumptions:** misuse is likely at scale; the affected person may not be a user.
- **Tradeoffs:** controls add friction for honest users.
- **Failure modes:** designing for the owner only; adding a control with no detection; hiding residual risk.
- **Worked example:** IQ-C22-06.

Who	How	Harm	Control	Residual
-----	-----	------	---------	----------

4. Answer architecture

The system design answer, for 45 minutes

Use the time table in C22.2. At each step produce the artifact in FW-C22-01.

Pacing rules. If the interviewer gives requirements or scale, spend the saved time on the deep dive and on failure and abuse. If you run behind, keep the requirements card, the estimate with one decision, the main flow and the top failure or abuse case. Treat questions as steering.

How to say it. Be explicit about assumptions: "I'll assume 10 million daily users, about 20 actions each; tell me if that's far off." State a number and the decision it drives: "That's about 11,600 writes a second at peak, which makes me want to ask whether one database is enough."

How to close. A 60-second executive summary, then an invitation: "In short: [user and problem]; the design is [three parts]; the main tradeoff is [dials], and I chose [option] because [user reason]. The biggest risks are [failure and abuse], mitigated by [controls]. I'd launch to [stage] and measure [metric]. The first thing I would validate with engineering is [question]."

Type: design a notification, feed or messaging system

Estimate fan-out and peak; choose push or pull; design preferences, deduplication and quiet hours; cover provider limits and failure.

Type: design a system that touches privacy or safety (location, devices, messages)

Lead with the abuse-case pass and the privacy design; then the flow and the failure modes.

Type: design a payments or irreversible-action flow

Lead with idempotency, confirmation of outcome, reconciliation and consistency.

Type: design for a spike (a launch, a sale, a live event)

Lead with the peak estimate, elasticity, queueing and load shedding, and graceful degradation.

Type: explain a design to a non-technical audience

Use the executive one-pager: user outcome, risk, cost, decision.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"How big is this?"	Scale	State the formula and the peak factor; name the decision
"Walk me through a write."	Data flow	Number the steps; say what is synchronous
"Why not just use X?"	Technology theater test	Say what problem X solves here and what it costs
"What if that fails?"	Failure mode	Say what the user sees and how it degrades
"How could someone abuse this?"	Abuse case	Name who, how, harm, and a control
"Can this be stale?"	Consistency	Tie the answer to the cost of a wrong value
"Explain it to the CEO."	Communication	User outcome, risk, cost, decision

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption (invented). The arithmetic was computed. Each is a design at PM depth following the spine (FW-C22-01), written as notes you could say aloud.

Worked example 1 — A notifications system (Illustrative)

Clarify and require (FW-C22-02). A social app sends notifications (a comment, a message, a friend request) to its users by push and email. Users: people who receive them and the product teams that trigger them. Must do: deliver the right notification on the right channel, respect each user's preferences and quiet hours, and never send the same notification twice. Latency: most within 5 seconds, digests on a schedule. Durability: it is acceptable to lose a low-value like notification occasionally, never a security alert. Privacy: no message content in logs. Non-goal: a template editor for marketers.

Estimate (FW-C22-03). 20,000,000 daily users (invented) receive an average of 8 notifications a day: 160,000,000 a day, $160,000,000 \div 86,400 =$ about 1,850 a second on average. Peak factor 5 (evening, and events): about 9,260 a second. Each notification record is about 300 bytes: 48 GB a day, and 1.44 TB for 30 days of history. These numbers say the volume is large for one worker and ordinary for a queue-based design, and that the push provider's rate limit (say 5,000 a second per connection) matters at peak.

Sketch (FW-C22-04). *Write path*: an event ("Ana commented on your post") arrives at the notification service, which (1) looks up the recipient's preferences and quiet hours, (2) checks for a duplicate using an event identifier, (3) decides whether to send now, batch ("Ana and 4 others commented") or hold until morning, (4) writes the record and puts a delivery

task on a queue. *Delivery*: channel workers (push, email) take tasks from the queue, call the provider, record success or failure, and retry on failure with growing delays. *Read path*: the app's notification list reads recent records from the store.

Tradeoffs (FW-C22-05). Latency against volume: batching reduces the number of pushes (less annoyance, lower cost) and delays some by seconds or minutes; choose short batching windows for messages, long for likes. Reliability against duplicates: a retry can send twice, so each delivery uses an idempotency key (C21.3). Cost: email costs more per message than push; prefer push where the user allows it.

Failure modes (FW-C22-06).

Component	Failure	User sees	Degradation	Signal
Push provider	Slow or down	Late or no push	Queue holds tasks; retry with backoff; fall back to in-app badge	Delivery success rate; queue age
Queue	Backlog at peak	Delayed notifications	Shed low-priority (likes) first; keep security alerts	Oldest task age
Preferences store	Unavailable	Possible wrong channel	Default to the most conservative setting (no email)	Lookup error rate
Duplicate event	Producer retries	Same notification twice	Event-id dedupe	Duplicate rate (should be near zero)

Abuse (FW-C22-08). A bot mass-mentions users to spam them; a malicious account sends harassing messages. Controls: rate limits per sender, a daily cap per recipient from the same sender, user controls to mute, and reporting.

Measures and rollout. Primary measure: the share of notifications delivered within 5 seconds (target 99%); guardrails: opt-out rate, uninstalls, duplicate rate, complaints. Launch to 1% of users and watch the guardrails (C17.2). *The first thing to validate with engineering*: whether the provider's rate limit holds at 9,260 a second.

What this shows. The estimate located the constraint (peak against provider limits), and the abuse and duplicate controls came from asking what could go wrong for the person receiving.

Worked example 2 — A news feed: fan-out on write or on read (Illustrative)

Requirements. A feed shows the recent posts of the accounts a user follows. Must do: show new posts within seconds, load fast on open. Freshness: seconds are fine; an hour is not. Scale (invented): 50,000,000 daily users, each opening the feed 10 times a day; 10,000,000 posts a day; an average account has 200 followers, and a few accounts have 20,000,000. Latency target: the 95th percentile of a feed open under 300 ms.

Two designs. *Fan-out on read (pull):* when a user opens the feed, the system fetches recent posts from each of the 200 accounts they follow and merges them. *Fan-out on write (push):* when someone posts, the system writes the post into the feed list of each follower in advance; opening the feed just reads one list.

The numbers (FW-C22-03). Feed opens: $50,000,000 \times 10 = 500,000,000$ a day, about 5,800 a second, and 23,100 at a peak factor of 4. *Pull:* each open fetches from 200 accounts, so peak load is about $200 \times 23,100 = 4.6$ million reads a second. *Push:* each post is written to 200 feeds: $10,000,000 \times 200 = 2,000,000,000$ writes a day, about 23,100 a second, and 92,600 at the peak factor of 4. Push costs 92,600 writes a second at peak but makes each open a single read; pull costs 4.6 million reads a second at peak. Since reads outnumber posts 50 to 1, push is far cheaper where it works.

The celebrity problem. An account with 20,000,000 followers posting once creates 20,000,000 writes. At 10,000 writes a second for that job, that takes $20,000,000 \div 10,000 = 2,000$ seconds, about 33 minutes, before the last follower sees it, and it competes with everything else.

Hybrid decision (FW-C22-05). Push for ordinary accounts (under about 100,000 followers); for the few large accounts, do not fan out: at read time, merge their recent posts into the precomputed feed. This keeps writes bounded and adds a few reads for users who follow large accounts. Freshness: ordinary posts appear in followers' feeds within seconds; posts from large accounts appear at the next feed open. Ranking (which posts rise to the top) is a separate step (C23).

Failure modes. The feed store is down: show a cached copy of the last feed with a "reconnecting" banner. A write backlog after a burst: feeds lag; show posts in time order as they arrive. A user unfollows: the old posts remain in their precomputed feed until cleaned; remove on read by checking the current follow list. Eventual consistency is acceptable here: a few seconds' difference between users is harmless.

Abuse. Spam accounts follow many users and post links; limit follows and posts per hour, and detect new accounts posting identical content.

What this shows. The estimate showed a hundredfold difference between two designs, and the exception (very large accounts) was found by checking the extreme.

Worked example 3 — Search over listings in a marketplace (Illustrative)

Requirements. Buyers search 20,000,000 listings by words and filters (category, price, location). Must do: return relevant results fast, tolerate spelling mistakes, show a new listing within about a minute, and not show a sold item as available. Latency: a 95th percentile under 300 ms. Scale (invented): 5,000,000 daily users, 6 searches each. Non-goals: personalized ranking in the first version (C23).

Estimate. $5,000,000 \times 6 = 30,000,000$ searches a day, about 350 a second on average, and about 1,390 at a peak factor of 4. The index holds 20,000,000 listings of about 2 KB of searchable text each: 40 GB, small enough to keep in memory on a few machines. If each full copy of the index can serve about 300 searches a second, the peak needs $1,390 \div 300 = 4.6$, so 5 copies: about 200 GB of memory in total.

Latency budget (FW-C21-03, 95th percentile). Network and connection 80 ms; API and checks 40 ms; search service 120 ms; filling in details (prices, photos) from a cache 40 ms; rendering 20 ms: 300 ms in total. If a search takes 120 ms at the median and 400 ms at the tail, the budget fails at the tail, so the design must watch the tail.

Sketch. *Write path:* a seller creates or changes a listing: the service writes it to the database (the record of truth), then puts an "index this" message on a queue; an indexer updates the search index within seconds (freshness target 60 seconds). *Read path:* the app sends the query to the search service, which understands the words (including common misspellings), uses the index to find and rank matching listing identifiers, then reads details from a cache, and returns results.

Tradeoffs. Freshness against cost: updating the index per change is fresh and costly; batching every 30 seconds is cheaper and meets the 60-second target. Staleness of availability: a listing can sell between indexing and a buyer's click, so the system checks availability when the buyer opens the item (a read from the database), accepting a few seconds of staleness in the list.

Failure and abuse. If the index is behind, show results with no newest listings, and alert when lag exceeds two minutes. If the search service is down, fall back to browsing by category from the cache. Abuse: scraping (rate limits), spam and fake listings (detection at creation, reporting).

Measures. Searches with a click (success), zero-result rate (C16 CS-C16-01 used the same measure), 95th-percentile latency, index lag. Staged rollout of any index change (C17).

What this shows. Estimating the index size showed that the problem is small enough for memory, which simplified the design, and the latency budget showed where the tail would hurt.

Worked example 4 — A payments flow with retries and webhooks (Illustrative)

Setting: a payments service of the kind Stripe offers. No fact about any real service is claimed.

Requirements. A merchant's customer pays; the service charges the card and tells the merchant. Must never: charge twice, or lose a record of a charge. Durability: a payment record is permanent. Consistency: strong for the ledger. Latency: a result within about 3 seconds. Scale (invented): 1,000,000 payments a day, about 12 a second on average and about 116 at a peak factor of 10.

Sketch (FW-C22-04). *Write path:* (1) the merchant's server sends a "create payment" request with an idempotency key; (2) the payment service records the payment as "processing" in the ledger (append-only), (3) calls the card network or bank partner, (4) records "succeeded" or "failed," (5) responds, and (6) sends a webhook to the merchant's web address to confirm. *Retry safety:* if the merchant's request is repeated with the same key, the service returns the first result and does not charge again (C21.3).

The hard case: unknown outcomes. The call to the partner times out; the service does not know whether the charge happened. Suppose 0.5% of payments end this way: 5,000 a day. The system must not guess. It marks the payment "unknown," asks the partner for its status shortly afterward, and records the answer; a daily reconciliation compares its ledger with the partner's report and flags differences for follow-up. The user sees "payment processing" and not a false success or failure.

Webhooks. A webhook is a message to the merchant's server saying what happened. The service delivers each at least once (so a merchant's brief outage does not lose events), retrying at growing intervals (for example, 1 minute, 5 minutes, 30 minutes, 2 hours, 12 hours and 24 hours: six attempts over a day), so a merchant may receive the same event twice, and events may arrive out of order. Merchants must therefore handle duplicates using the event identifier and, for ordering, read the current state from the service. The service signs each webhook so that merchants can verify it came from the service, and offers a way to replay missed events.

Tradeoffs (FW-C22-05). Consistency against availability: for the ledger choose strong consistency (a payment must never be recorded in two ways) and accept that during a database failure the service stops taking payments rather than risk errors. Latency against safety: confirming with the partner before answering adds time and prevents false successes.

Abuse. Card testing: attackers try many small charges to find valid cards; defend with limits per card, per IP and per merchant, and velocity checks (a sudden surge of failures).

Failure table.

Component	Failure	User sees	Degradation	Signal
Partner	Timeout	"Processing"	Mark unknown; poll; reconcile	Unknown-outcome rate
Ledger database	Unavailable	Cannot pay, clear message	Stop accepting; no partial writes	Write error rate
Merchant endpoint	Down	Merchant notified late	Retry with backoff; replay	Webhook failure age

Measures. Payment success rate, rate of unknown outcomes and time to resolve, duplicate charges (target zero), webhook delivery within 1 minute. Reliability target per FW-C21-06.

What this shows. The design is driven by the cases in which the outcome is unknown, and by the promises the service makes to others (at least once, signed, replayable).

Worked example 5 — A link-shortening service (Illustrative)

Requirements. Users turn long web addresses into short ones; clicking a short link redirects to the original. Must do: redirect fast and reliably, never send people to the wrong place. Scale (invented): 100,000,000 new links a month; reads 100 times more than writes. Non-goals: custom domains in version 1.

Estimate. New links: $100,000,000 \div (30 \times 86,400) =$ about 39 a second. Redirects: 100 times that, about 3,860 a second on average and 19,300 at a peak factor of 5. Storage: $100,000,000 \text{ links} \times 500 \text{ bytes} = 50 \text{ GB a month, } 600 \text{ GB a year, } 3 \text{ TB over five years.}$ Short codes: with 7 characters from 62 symbols (letters and digits), there are $62^7 = 3.5$ trillion codes; five years of links is 6 billion, about 0.17% of the space, so collisions are rare and manageable. With 6 characters (56.8 billion codes) it would also be enough, at about 10% occupancy over five years.

Sketch. *Write:* the service generates a code, checks it is unused, stores the code and the target in the database. *Read:* the redirect service looks up the code, first in a cache, then in the database, and returns a redirect. Because a small share of links gets most clicks, a cache of the hottest links answers most requests. If the hottest 1% of links draw 80% of traffic (an assumption to test), then of the 1,200,000,000 links created in a year only about 12,000,000 need to be cached, at 500 bytes each about 6 GB, which fits in memory.

Tradeoffs. Redirect type: a permanent redirect lets browsers remember it and lowers load, but later clicks do not reach the service, so counts of clicks are undercounted; a temporary redirect keeps counts accurate and costs more load. Choose by whether click analytics is a product feature. Code generation: sequential codes are simple and guessable (anyone can enumerate other people's links); random codes are not guessable; for private links use random codes.

Failure. If the database is unavailable, serve from the cache (links keep working; new links wait). A wrong redirect is the worst failure, so changes to stored targets are logged and limited.

Abuse. Short links hide destinations, so they are used for phishing and malware. Controls: scan targets at creation and periodically, a report-and-takedown path, warnings for flagged links, limits on creation by new accounts, and, for high-risk categories, an interstitial page that shows the destination.

What this shows. The estimate said that reads dominate, so a cache is the main lever, and the abuse pass found the real product risk.

6. Baseline, Strong, Standout

Prompt: "Design a system that sends a daily summary email to 5 million users."

Baseline answer

"I'd have a service that gets the list of users, builds each email and sends it using an email provider. I'd use a queue to handle the volume and a database to store the users."

What is wrong: technology named, no requirements, no numbers, no sense of timing, failure or the user's view.

Strong answer

"First I'd ask what the summary contains, how fresh it must be, when users expect it, and whether they can opt out. I'd assume 5 million users and one email a day.

"If all 5 million emails go out at once, that is a burst. A provider might allow, say, 500 a second, so 5 million would take about 2.8 hours; the last people would get a morning email at lunchtime. So I'd send by each user's local time, which spreads the load across 24 hours, and cap the rate. I'd use a queue so that sending can retry, make each send idempotent so nobody gets two, honor unsubscribes, and track bounces. I'd measure delivery time, open rate and unsubscribes."

Standout answer

Everything in the Strong answer, plus:

"I'd put numbers on the burst: $5,000,000 \div 500 = 10,000$ seconds, 2.8 hours. With 24 time-zone buckets the busiest hour might hold 15% of users, 750,000 in an hour, about 208 a second, comfortably under the cap; adding random spread inside the hour smooths it. The email also causes a burst of app traffic: if 10% of recipients open the app within 10 minutes, that is 500,000 opens in 600 seconds, over 830 extra requests a second on top of normal traffic, so I'd warn the app team and cache the landing page.

"I'd think about failure: if the email provider is down, the queue holds tasks and retries; if it is down for hours I'd drop to a later batch and skip the stale summary rather than send yesterday's news. A user who gets a digest twice is worse than one who gets none, so I'd prefer at-most-once delivery for the summary and at-least-once for security emails.

"Privacy and abuse: no sensitive content in an email (it may be read on a lock screen or shared), a one-click unsubscribe, and rate limits so the system cannot be used to send arbitrary email.

"I'd validate first with the email provider the actual sending rate and with deliverability experts that bunching 24 buckets does not harm sender reputation. What would change my design: if the summary must contain data that changes by the minute, I'd render at send time and accept the cost."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Asks requirements and estimates the burst	Computes the burst, the spread and the downstream app spike
Spreads sending by time zone and uses a queue with retries	States delivery guarantees by email type and a failure policy
Honors opt-out and measures	Covers privacy, abuse, and what to validate first

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats change [as of 2026-10].

Company lens: Google [as of 2026-10]. Google publishes extensively on large-scale systems and site reliability, and writes design documents before major work Public [V: Google SRE book; design documents not checked]. Candidates commonly report design prompts at large scale, such as sharing, maps or search Inferred. Practical consequence: lead with scale and the latency tail, say what is cached and how fresh it must be, and ask engineering about the riskiest part.

Company lens: Meta [as of 2026-10]. Meta describes serving billions of people with feeds, messaging and notifications Public [V: Meta Form 10-K for 2025: 3.58 billion family daily active people]. Candidates commonly report prompts on feed, stories, notifications and messaging systems Inferred. Practical consequence: reason about fan-out, bursty load, ephemeral content and expiry, and abuse at scale.

Company lens: Microsoft [as of 2026-10]. Microsoft's products serve enterprises that require permissions, auditing and compliance, including real-time collaboration in documents Public [V: Microsoft docs: real-time co-authoring in Word, Excel and PowerPoint, including files protected by sensitivity labels that admins enable in Microsoft Purview; Microsoft 365 Security and Compliance named in the 10-K; audit logging not separately checked]. Candidates commonly report design prompts for collaborative or enterprise products Inferred. Practical consequence: cover permissions, audit logs, conflict handling when people edit together, offline behavior and data residency.

Company lens: Amazon [as of 2026-10]. Amazon's leadership principles include Customer Obsession and Dive Deep, and it runs both a large store and a cloud platform Public [V: Amazon's Leadership Principles, as cited by several summaries]. Candidates commonly report prompts on orders, fulfillment tracking and reliability at peak events Inferred. Practical consequence: start from the customer's question ("where is my order?"), size the peak, and describe how the system behaves when partners send late or duplicate events.

Company lens: Apple [as of 2026-10]. Apple has described designing features so that sensitive data stays on the device or is protected from the company itself, and runs a network that helps people find lost devices Public [V: Apple privacy pages and Find My & Privacy page (end-to-end encrypted Find My network)]. Candidates commonly report design prompts that combine privacy with utility Inferred. Practical consequence: lead with the privacy design and the abuse cases, and say what you give up for privacy.

Company lens: Netflix [as of 2026-10]. Netflix has published on delivering video through a content network placed close to viewers and on testing resilience by injecting failures Public [V: Netflix Tech Blog posts on experimentation and resilience testing]. Candidates commonly report prompts on playback, startup time and graceful degradation Inferred. Practical consequence: reason about time to first frame, adaptive quality, regional capacity and what the viewer sees when a part fails.

Company lens: Stripe [as of 2026-10]. Stripe's documentation describes idempotency keys, webhooks with retries and signatures, and API versioning Public [V: Stripe API documentation on idempotency, webhooks and versioning]. Candidates report design prompts on payments and developer-facing delivery Inferred. Practical consequence: design for unknown outcomes, at-least-once delivery, replay and reconciliation, and treat the API contract as a product.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Starting with technology	A tool named before any requirement	Clarify users, job and constraints first
2	No numbers	"It scales"	Estimate average and peak, and use them to decide
3	Averages for design	No peak factor	State a peak factor and why
4	Boxes with no flow	A diagram with unlabeled arrows	Number the write and read paths
5	Ignoring the user's view of failure	"It retries"	Say what the user sees in each failure
6	No abuse pass on a feature touching people	Designed only for the owner	Run the abuse-case pass
7	One consistency setting for everything	"Everything strongly consistent"	Match consistency to the cost of a wrong value

#	Mistake	How to detect it	Fix
8	Duplicates ignored	Retries with no idempotency	Make actions idempotent; dedupe events
9	Over-design for version 1	Many services for a small product	Start simple; state what would trigger more
10	No summary for the audience	A design only engineers can follow	Prepare the executive version

9. Mental models applied

Four models, each tied to a decision in this chapter. [[pareto-principle]] is from C03.

Little's law (MM-littles-law). The average number of items in a system equals the arrival rate times the average time each spends in it [V: Little's law, $L = \lambda \times W$, as listed in software-engineering law references]. If a service handles 1,000 requests a second and each takes 0.2 seconds, about 200 are in progress at once, which sizes the number of connections or workers. It turns "how many servers?" into arithmetic.

The hot set (MM-hot-set, see [[pareto-principle]], C03). A small share of items draws most requests, which is why caches work. Always ask what the hot set is and whether it fits in memory (Worked examples 3 and 5).

Everything fails, all the time (MM-everything-fails). A line associated with Amazon's chief technology officer: in large systems something is always failing, so design for it [V: Werner Vogels, A Second Conversation with Werner Vogels, ACM, 2021: Everything fails, all the time; his current title was not checked]. Treat failure as a normal condition, and ask of every component what the user sees when it fails.

Orders of magnitude of delay (MM-latency-numbers). Reading from memory is about a thousand times faster than a network trip within a data center, which is far faster than a trip across an ocean [V: Dean, Numbers Everyone Should Know (talk slides)]. Knowing the ladder tells you why caches, nearby copies and asynchronous work matter, and why a design with many sequential network calls is slow.

Engineering, design and data-science perspectives

- **Engineering.** Engineers will refine your estimate and sketch; give them assumptions and open questions, not choices of technology. Ask which part is hardest, what has been done before, and what the plan is at ten times the load.
- **Design and research.** Designers own what the user sees during delay, failure and degradation (C10.5). Share the failure table so that loading, offline and error states are designed, not left to chance.
- **Data science.** They define the measures and the instrumentation: delivery rates, latency percentiles, duplicate rates, abuse signals. Agree the events the system must log, and how privacy limits them.

- **Security, privacy, trust and safety.** They lead the abuse-case pass and the privacy design. Bring them in before the sketch is final, and let them name the abuse cases you did not think of.

Checklist: before you present a design

1. Did I ask who uses it, the core job, scale and constraints, and state my assumptions?
2. Did I estimate average and peak load and storage, and say what decision each number drives?
3. Can I walk a write and a read through the sketch, saying what is synchronous?
4. Did I name the tradeoffs and tie each choice to the user?
5. Do I know what the user sees when each part fails, and how it degrades?
6. Did I run an abuse-case pass if the system touches people, location, messages or money?
7. Did I choose consistency and delivery guarantees by the cost of a wrong or duplicate result?
8. Can I give a 60-second executive summary and a list of open questions for engineering?

10. Case studies

CS-C22-01 · The 9 a.m. digest that arrived at lunch · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A news app has 5,000,000 subscribers to a daily digest email. A new engineer schedules it for 09:00 server time, which is 09:00 UTC. The team assumes the email provider can handle the volume.

Constraints. The email provider allows about 500 sends a second. Subscribers are spread across many time zones. The app normally handles about 1,000 requests a second.

Data. On the first day, the digests begin at 09:00 UTC and finish at about 11:47 UTC. For subscribers in Europe the 09:00 UTC digest arrives in the middle of the morning, and for subscribers in the Americas in the early morning, hours before they wake. Support receives complaints that the "morning" digest comes at lunchtime for some and at 4 a.m. for others. At the moment the first batch lands, about 10% of recipients open the app within ten minutes.

Problem definition. As an observable problem: "Sending 5,000,000 emails at one moment exceeded the provider's rate, delayed most of them by up to nearly three hours, ignored where users live, and produced a spike of app traffic."

Analysis. *The burst:* $5,000,000 \div 500 = 10,000$ seconds, about 2.8 hours, which matches the observed finish at 11:47. *The user's view:* a "morning" email at the wrong local time. *The knock-on spike:* 10% of 5,000,000 is 500,000 opens in 600 seconds, about 833 requests a

second on top of the usual 1,000, an 83% jump in load. Sending by local time spreads the load: if the busiest hour holds 15% of subscribers, that is 750,000 in an hour, 208 a second, well under the cap, and the app traffic follows the same smooth pattern.

Options.

Option	Optimizes	Costs
A. Raise the provider's cap	Finish the burst faster	Cost; the wrong local times remain
B. Send by each subscriber's local time, with random spread inside the hour	Right time for the user; low peak	A scheduling job; the content is built per time zone
C. Send all at once earlier, at 05:00 UTC	Done before most European mornings	Americas still wrong; still a burst

Tradeoffs. B costs a modest scheduling change and solves both the burst and the local-time problem. A and C treat only the rate.

Decision. Choose B. The digest is built once a day per time-zone bucket and sent at 07:30 local time with a random spread of ± 15 minutes. Primary measure: the share of digests delivered within 30 minutes of the subscriber's target time; guardrails: unsubscribes, duplicate sends (target zero), app 95th-percentile latency in the hour after sends.

Outcome. In this fictional scenario, the peak send rate fell to about 210 a second, 98% of digests arrived within 30 minutes of the target, and the app's load after sends stayed within 15% of normal (invented).

Alternative outcomes. If the app's content changed hourly, building per bucket could show older news in later buckets, and the team would build at send time. If the provider's rate had been 100 a second, the busiest bucket would still queue, and the team would spread across more hours. The lesson: size the burst and put the user's clock into the design.

CS-C22-02 · Pokémon GO, 2016: a launch that outran its worst case · Verified · Advanced

Sources (types): Google Cloud's account of the launch by Luke Stone, then director of customer reliability engineering at Google Cloud ("Bringing Pokémon GO to life on Google Cloud," Google Cloud blog, 29 September 2016), read in full (primary); and trade and general press coverage that quotes it. The analysis is the author's. The post does not reveal Niantic's internal estimates or decisions beyond what Google reported. Re-read in October 2026.

Situation. Niantic launched Pokémon GO on 6 July 2016, first in Australia and New Zealand, then in the United States the next day, with a Japan launch later. Google Cloud reported that the teams had targeted $1\times$ player traffic with a worst-case estimate of roughly $5\times$ the target, and that popularity quickly surged player traffic to $50\times$ the target, ten times the worst-case estimate; Cloud Datastore, the game's primary database, became a direct proxy for its overall popularity, and the post's graph shows the three lines [V: Google Cloud blog, 2016]. Within 15 minutes of the launch in Australia and New Zealand, player traffic surged well past Niantic's expectations, and Niantic phoned Google's customer reliability engineers for

reinforcements ahead of the US launch planned for the next day [V: same]. Google said not everything went smoothly at launch and that Niantic and Google engineers addressed problems in sequence. Before the Japan launch they upgraded to a newer version of Google Container Engine that would allow more than a thousand additional nodes in the cluster, and replaced the network load balancer with an HTTP/S load balancer; Google said the lessons from the US launch, including generous capacity provisioning, paid off when the game launched without incident in Japan, where the number of new users signing up tripled that of the US launch two weeks earlier [V: same]. Google added that this was the largest Kubernetes deployment on Container Engine to that date, with many tens of thousands of cores provisioned [V: same].

Constraints. A game with an unprecedented, hard-to-predict audience; a launch in stages by country, with the next market a day away; a design on a cloud platform that could add capacity, subject to how quickly it could be added; and a dependency on services (the database, the container platform, load balancing) that had never been run at that load.

Data. As reported: the target, the worst case (about $5\times$) and the actual ($50\times$) in relative terms; the 15-minute window; the staged launch order; the escalation to Google; the platform upgrade that allowed more than a thousand additional nodes and the load-balancer change; the smooth Japan launch with three times as many new sign-ups as the US launch. Absolute traffic numbers are not in the sources used here.

Problem definition. *The author's reading.* The estimate for a viral product was off by an order of magnitude even in its worst case, so the system's ability to add capacity quickly mattered more than the accuracy of the estimate. As an observable problem: "Demand ran at ten times the worst case within minutes of launch."

Analysis. *The author's analysis, through this chapter's tools.*

The estimate and its limits (FW-C22-03). A planned worst case of $5\times$ is a conventional safety margin for a product with a known audience. For a product that can go viral, the distribution of outcomes has a long tail, and a point estimate, even a pessimistic one, can be wrong by a factor of ten. The better question than "how big?" is "how fast can we add capacity, and what happens if we cannot?"

Elasticity and the real bottleneck. The sources point to constraints that were not the number of servers alone: the rate at which containers could be provisioned on demand, a database service whose transaction rate tracked popularity, and load balancing. Google's response included changing the platform's provisioning, which is a reminder that a system is limited by its slowest-scaling part, and that the part may belong to a supplier.

Staged launch as a risk control (C17.2). Launching country by country gave the teams a day between markets to fix problems. That ordering, whether or not it was chosen for that reason (the sources do not say), worked as a staged rollout: the smaller markets absorbed the surprise, and capacity was added before larger ones. The Japan launch, with more downloads, was smoother.

Graceful degradation (FW-C22-06). The sources describe connection problems at launch and sequential fixes; they do not describe how the game degraded for users in detail. A design question for any such launch is what a player sees when the backend is saturated (a queue or a message, not an error), and whether sign-ups can be metered.

Escalation paths. A direct line to the platform provider's reliability team, available ahead of the next launch, is part of the system.

What the evidence does not show. The internal capacity plan, the cost, the exact failures and their duration, and how players experienced them minute by minute. The case supports an analysis of planning and elasticity, not a judgment of any team.

Options. *The author's reconstruction of choices a team could face; not a record of Niantic's deliberations.*

Option	Optimizes	Costs
A. Size the system to the target and add capacity reactively	Cost	Outages if demand exceeds target; slow recovery
B. Size to the worst-case estimate (5×)	Resilience to a bad case	Cost; still short if reality is 50×
C. Size for elasticity: a staged launch by country, capacity added quickly, direct escalation to the provider, load shedding or metered sign-ups	Handling an uncertain tail	Operational complexity; dependence on the provider's scaling

Tradeoffs. B buys a larger fixed margin than A and is still wrong by ten times; C accepts uncertainty and invests in the ability to respond.

Decision. As documented, the launch was staged by country, with capacity added in response and Google's reliability engineers engaged.

Outcome. Reported by Google: stability issues at launch that engineers addressed in sequence, and a Japan launch without incident two weeks after the US launch [V: Google Cloud blog, 2016]. Later operations are outside the sources used here.

Alternative outcomes. *Inference.* Had the first launch been worldwide at once, the shortfall would have hit every market simultaneously. Had sign-ups been metered (a waiting room), fewer players would have been admitted and the experience of those admitted would have been better, at the cost of turning some away. A reader should avoid hindsight: a worst case of 5× was a sensible plan until reality exceeded it. The method to carry away: for products with uncertain demand, plan for how fast you can scale and what you will shed, stage the launch so that early markets absorb the surprise, identify the slowest-scaling component, and arrange escalation before you need it.

11. Interview questions and model answers

Ten questions. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each ends with follow-ups and answers, because a design question is rarely a single question. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. The answers are fuller than a first pass in a real round.

IQ-C22-01 · Beginner · system design · SD

Prompt. What does a PM need to show in a system design question, and how do you start one?

Rubric.

- **Baseline:** "I'd name the technologies I'd use."
- **Strong:** Clarifies requirements, estimates scale, sketches the flow and covers failure.
- **Standout:** Explains the PM-level bar, states assumptions and the decisions the numbers drive, covers abuse and privacy, and closes with an executive summary and questions for engineering.

Model answer. The question is not whether I can build the system; it is whether I can turn an open request into a bounded problem, size it, sketch how the pieces fit, weigh tradeoffs, and say what breaks. So I start with questions, not boxes.

I begin by saying my plan: "I'll clarify who it's for and what it must do, estimate the scale, sketch the main flows, go deep on the part that matters most, then cover failure and abuse, and summarize." Then I ask the few questions whose answers change the design: who the users are and the core job; the scale (users and actions a day); how fast and how fresh results must be; what must never be lost or duplicated; what the privacy or regulatory constraints are; and what is out of scope.

Next I estimate: daily users times actions, divided by 86,400 seconds, gives an average rate; a peak factor gives the peak. I say what decision each number drives. Then I draw a handful of boxes (clients, entry point, a few services, data stores, a queue) and walk one write and one read through them, saying where the user waits.

I name the main tradeoff and choose with the user in mind, say what the user sees when something fails, and run a quick abuse pass if the system touches people or money. I finish with a 60-second summary for an executive and the one thing I would validate with engineering first.

Follow-ups.

- **Q.** The interviewer says, "Just pick the technologies." What do you do? **A.** "I give a choice with a reason tied to the access pattern, and say where I would defer to engineering: 'Records that must not be lost and are looked up by user suggest a relational database; I would ask engineering to confirm it handles about 9,000 writes a second.' Naming a tool without a reason sounds like memorization; naming the need and the question shows judgment."

- **Q.** How do you decide how much detail to give? **A.** "Enough that an engineer could start and an executive could follow: the user's job, the numbers that matter, the main flow and the main risks. I go deep on one or two parts where the tradeoff is real and leave the rest at the box level, and I ask the interviewer where they want depth."

IQ-C22-02 · Beginner · system design · SD

Prompt. Estimate the traffic, storage and bandwidth for a photo-sharing app with 10 million daily users.

Rubric.

- **Baseline:** One large number with no method.
- **Strong:** Uses $\text{users} \times \text{actions} \div 86,400$, a peak factor, item sizes and retention.
- **Standout:** States each assumption, separates reads and writes, uses the numbers to draw conclusions (CDN, storage cost), and sanity-checks.

Model answer. I'll state assumptions and use the sheet. Suppose each daily user uploads 1.5 photos and views 40.

Uploads. $10,000,000 \times 1.5 = 15,000,000$ photos a day, $15,000,000 \div 86,400 =$ about 174 a second; at a peak factor of 4, about 700 a second. That is a low write rate: ordinary for a queue-based design.

Storage. Suppose the original averages 2.5 MB and resized versions add about 0.3 MB: 2.8 MB a photo. $15,000,000 \times 2.8 \text{ MB} = 42 \text{ TB}$ a day, about 15 PB a year. Storage is the big number: it will dominate cost, so I would ask about compression, a retention policy, and cheaper storage for old photos.

Views. $10,000,000 \times 40 = 400,000,000$ views a day, about 4,630 a second, and 18,500 at the peak factor of 4.

Bandwidth. If a viewed photo averages 200 KB, 4,630 views a second is about 925 MB a second, about 7.4 Gbps on average and about 30 Gbps at peak.

What the numbers say. Writes are modest; reads and bandwidth are large, so serve photos from a content delivery network near users, with a cache for hot items. Storage grows by about 42 TB a day, so cost per user matters: 42 TB a day over 10 million users is 4.2 MB per user per day. Sanity check: 15 PB a year is large but plausible for a large photo product.

Follow-ups.

- **Q.** How would you cut storage? **A.** "Store fewer large versions and generate others on demand, compress more aggressively for old photos, deduplicate identical uploads, and move photos nobody has viewed in a year to cheaper storage with a slower retrieval. Each has a user cost (quality, delay), so I'd test with a small group."
- **Q.** Why a peak factor of 4? **A.** "Usage clusters in the evening and on weekends, so the busiest hour carries more than an average hour. For a photo app I'd start at three to five and ask engineering for the real ratio from similar products; for an event-driven product it would be higher."

IQ-C22-03 · Intermediate · system design · SD · Google

Prompt. Design a shared photo album service for families.

Rubric.

- **Baseline:** "Upload photos and share a link."
- **Strong:** Covers permissions, storage ownership, notifications, and scale.
- **Standout:** Raises link-sharing risk and revocation, deletion semantics, abuse, cost per user, and degradation.

Model answer. The setting is general; I know such products only from public information, and nothing here describes any real design. The job: family members add photos to an album and see everyone's photos, with notifications when something is new.

Requirements. Create an album, invite people, add and view photos, comment, get notified. Privacy: only invited people can see it, with the option of a link for relatives who do not have accounts. Durability: photos must not be lost. Freshness: new photos appear within seconds. Non-goals: editing tools.

Scale (invented). 20 million daily users; 5 million photos a day added to shared albums, each notifying 4 other members: 20 million notifications a day, about 230 a second; 30 million album opens a day, about 350 a second. Modest rates; storage and sharing rules are the design issues.

Sketch. An album is a record with a member list and permissions; photos go to object storage with thumbnails made by a worker; a "new photo" event goes on a queue; a notification service sends a push to members who opted in, batching bursts ("Ana added 12 photos"). Album opens read the member list, check permission, and serve thumbnails from a content network.

Decisions that matter. *Who owns the storage:* if a photo counts against the uploader's quota, uploading to a shared album has a cost to that person, so say so plainly. *Link sharing:* an unlisted link is convenient and can leak, so use long random links, let owners turn them off, and show who has viewed. *Deletion:* if the owner leaves or deletes the account, decide whether the album persists for members; I'd transfer ownership and warn first. *Removal:* when someone leaves, they lose access to the album but keep the photos they uploaded unless they delete them.

Failure and abuse. If thumbnails fail, show a placeholder and retry. If notifications are delayed, the album still shows new photos. Abuse: an unwanted person added to an album (limit invites, allow a recipient to decline and report), explicit content shared (reporting and scanning, with care for privacy).

Measures. Albums with two or more contributors, weekly photos added per album, notification opt-out rate, link-leak reports.

Company note. Google is the setting only: **Inferred** from candidate reports of sharing and photo design prompts **[unverified: candidate-reported]**. No Google data or design is claimed.

Follow-ups.

- **Q.** What happens when someone leaves the album? **A.** "They lose access to the shared view; photos they uploaded stay in the album unless they choose to take them back, and they are asked at the time. Notifications to them stop at once, and the removal is recorded so that they cannot be silently re-added."
- **Q.** How do you stop a link from leaking? **A.** "I can't fully stop it, so I limit the damage: random links that are hard to guess, an expiry option, a one-click 'turn off link,' a list of who opened it, and a prompt to review links that have been shared widely. Sensitive albums can require sign-in instead of a link."

IQ-C22-04 · Intermediate · system design · SD · Meta

Prompt. Design "Stories": photos and videos that disappear after 24 hours.

Rubric.

- **Baseline:** "Store the posts and delete after 24 hours."
- **Strong:** Handles expiry, views, fan-out, and storage.
- **Standout:** Separates read-time hiding from deletion, covers the view list and privacy, estimates load, and covers abuse and ephemerality promises.

Model answer. The setting is general; nothing here describes any real design. The job: people post short photos and videos that friends can view for 24 hours.

Requirements. Post a story; friends see it in a tray; the poster sees who viewed it; it disappears after 24 hours. Freshness: a new story appears in seconds. Privacy: only the audience the poster chose. Non-goals: permanent archives in version 1.

Scale (invented). 100 million daily users; 15% post a story a day with an average of 3 items: 45 million items a day, about 520 a second. Each item is viewed by about 40 people: 1.8 billion views a day, about 20,800 a second on average. A video averages 3 MB: 45 million × 3 MB = 135 TB a day of media.

Sketch. Write: upload media to object storage, create a story record (poster, audience, expiry time = posted time + 24 hours), publish an event. *Tray:* for each viewer, the set of friends with unexpired stories; because stories are few and short-lived, fan-out on read is workable: when a user opens the app, fetch the friends' latest stories, with a cache of "who has an active story." *View:* a view writes a record (viewer, story, time), appended through a queue and counted; the poster's view list reads from it.

Expiry. Hide at read time: every read checks that the expiry time has not passed, so a story disappears exactly at 24 hours regardless of cleanup. Delete the media and records later with a background job within a set period, which keeps storage bounded: media is held about 24 hours plus a grace period, so the live store holds about 135 TB, not a growing archive. Whether the platform keeps an archive for the poster or for safety review is a policy decision with privacy consequences, and should be stated.

Tradeoffs. View counts: exact counts cost writes; approximate counts that settle in seconds are fine, but the list of viewers should be accurate (people care who saw it). Consistency: eventual is fine for the tray; permissions must be strict at read time.

Failure and abuse. If the view-recording queue backs up, counts lag and are labeled "updating." Abuse: screenshots cannot be prevented, so promise only what the product can keep ("disappears from the app," not "can't be saved"); harassment through stories needs reporting, blocking and review that works within the 24-hour window.

Measures. Stories posted per user, viewing completion, time to appear (95th percentile under 5 seconds), reports per 1,000 stories handled within hours.

Company note. Meta is the setting only: `Inferred` from candidate reports of ephemeral-content design prompts `[unverified: candidate-reported]`. No Meta data or design is claimed.

Follow-ups.

- **Q.** Why not delete exactly at 24 hours? **A.** "Deleting precisely at the deadline across many machines is hard and wasteful. Checking the expiry time at read makes the disappearance exact for users, and a background sweep removes the data soon after. That gives correct behavior now and bounded storage later."
- **Q.** How do you handle the view list for a story seen by a million people? **A.** "Record each view through a queue, keep an approximate running count for display, and page the detailed list lazily, newest first, because nobody scrolls a million names. For small audiences the list is complete and exact."

IQ-C22-05 · Intermediate · system design · SD · Microsoft

Prompt. Design real-time co-editing for a shared document in an enterprise product.

Rubric.

- **Baseline:** "Save the document on a server and let people open it."
- **Strong:** Covers presence, conflict handling, latency, offline, and permissions.
- **Standout:** Explains conflict resolution choices in plain terms, protects against lost text, covers audit and compliance, and states what to ask engineering.

Model answer. The setting is general; nothing here describes any real design. The job: several people edit the same document at once and see each other's changes quickly, without losing work.

Requirements. Up to about 10 simultaneous editors; remote changes appear within about 200 ms; no lost text; works through brief connection loss; permissions (view, comment, edit) enforced; an audit trail for compliance; version history. Non-goals: editing the same cell of a table with perfect merging in version 1.

The central problem: conflicts. Two people type in the same sentence at the same moment. The system must decide how to combine edits. Options in plain terms: *last writer wins* (simple, loses someone's change), *locking* (only one editor per section, safe, frustrating), or

merging edits as operations (each change is recorded as an operation such as "insert 'the' at position 12", and a rule transforms concurrent operations so that everyone's document converges to the same result). Engineers use families of techniques for the third; I would not specify which, only the requirement: all copies converge and no one's typing disappears.

Sketch. Each editor's client sends operations to a collaboration service for that document, which orders them, applies them, and broadcasts them to the other editors; presence (cursors, who is online) travels on the same channel but is not stored. Operations are stored in a log, which gives version history and the audit trail.

Scale (invented). 10 active editors typing about 5 operations a second each: 50 operations a second per document. A document session is small; the scale comes from millions of documents, so route each document's session to one server and spread documents across servers.

Offline. Edits made offline are kept on the device and merged on reconnect. If the same sentence was changed on both sides, the system keeps both versions of the conflicting passage and flags it for a person to resolve, rather than guess. Suppose 2% of offline sessions have such a conflict (to be measured).

Permissions, audit, compliance. Permissions are checked on every operation, not just at open. The log records who changed what and when, retained per the customer's policy, with the ability to export for audit. Data residency: keep a customer's documents in their region.

Failure. If the collaboration service fails, clients keep working locally and reconnect; show "reconnecting" and never discard unsent edits. A server restart replays the log.

Questions for engineering. How do we guarantee convergence, what is our plan if one document has 200 editors, and how do we test conflict cases?

Company note. Microsoft is the setting only: `Inferred` from candidate reports of collaboration and enterprise design prompts `[unverified: candidate-reported]`. No Microsoft data or design is claimed.

Follow-ups.

- **Q.** What happens if two people edit the same sentence while offline? **A.** "On reconnect the system detects overlapping changes, keeps both versions of that passage, marks it visibly, and asks someone to choose. Silent overwrites lose work and destroy trust, so the product prefers a visible conflict to a quiet loss."
- **Q.** How do you keep an audit trail without storing every keystroke forever? **A.** "Group keystrokes into meaningful changes (a pause or a save creates one entry), keep fine detail for a short period, keep the grouped history for the retention period the customer's policy requires, and make the history exportable. The requirement to keep something and the cost of keeping everything are reconciled by grouping."

IQ-C22-06 · Hard · system design · SD · Apple

Prompt. Design, at PM level, a service that helps users find lost devices without exposing their location to others.

Rubric.

- **Baseline:** "Devices send their location to a server and the owner looks it up."
- **Strong:** Raises privacy, offline devices, and the stalking risk.
- **Standout:** Designs so that even the service cannot read locations, covers abuse cases in depth (including an abusive owner), estimates load, states what privacy costs, and plans a staged rollout.

Model answer. This is an Illustrative design at product depth; nothing here describes any real company's system. The job: an owner finds a lost device on a map, even when the device is offline, and no one else, including the company, learns where it is.

Requirements. Owner sees the last known or current location; works when the device has no internet; no one but the owner can learn the location; low battery cost; usable by non-experts. Must not: let a stranger track a person. Non-goals: tracking people.

The privacy design. A lost device has no connection of its own, so it relies on nearby devices of other people. The device broadcasts a short-range signal containing an identifier that changes frequently, so that it cannot be followed over time by an observer. Any nearby participating device that hears it reports the sighting, encrypted so that only the owner's other devices can decrypt it, together with its own location. The service stores and forwards encrypted reports it cannot read. Designs of this kind, using rotating identifiers and encrypted crowd-sourced reports, are publicly described by Apple and by Google [V: Apple Find My & Privacy page; Google Security Blog, April 2024; Google Find My Device help page]. The cost to say plainly: the company cannot help a user recover a location it cannot see, and cannot investigate misuse by reading data.

Scale (invented). 500 million devices; each reports about 20 sightings a day: 10 billion reports a day, about 116,000 a second, and about 350,000 at a peak factor of 3. At 100 bytes a report, 1 TB a day, kept for 7 days: 7 TB. Reports are small; the rate is large, so the service must accept writes cheaply, deduplicate (one report per identifier per 15 minutes per reporter), and expire quickly.

Abuse-case pass (FW-C22-08).

Who	How	Harm	Control	Residual
A stalker	Slips a small tracker into someone's bag or car	Following a person	Rotating identifiers and alerts to anyone whose phone detects an unknown tracker moving with them; an audible alert from the tracker after a period apart from its owner	Some devices cannot detect it; people without phones
An abusive partner or owner	Uses a shared device or account to follow a partner	Coercive control	Clear sharing indicators, notifications when location is shared, easy ways to stop sharing, safety guidance	A determined abuser with account access
A thief	Disables the device's finder	Loss of recovery	Requires owner credentials to turn it off or erase; lock the device to the account	Theft before setup

Who	How	Harm	Control	Residual
A bad actor	Floods reports to degrade the service	Cost, noise	Rate limits per reporter, signed reports from enrolled devices	Compromised devices

Failure and degradation. If no nearby device has sighted the item, show the last seen time and place, not a false current position. If the owner's account is unavailable, recovery through a trusted contact or an identity check, since support cannot read locations. If the service is down, the broadcasting still works and reports queue on devices.

Tradeoffs (FW-C22-05). Privacy against capability (no company access to locations versus no ability to help in a dispute), battery against detection delay (broadcast interval), and openness to third-party trackers against stalking risk (requiring them to meet the same alert standards).

Rollout and measures. Launch to a small group, then widen (C17.2), with guardrails on unwanted-tracking reports, battery impact, and false alerts. Success: recovered devices per lost device, time to find, unwanted-tracking alerts handled.

Company note. Apple is the setting only: **Public** that Apple operates a network for finding lost devices and has described its privacy approach, **Inferred** from candidate reports of privacy-focused design prompts [V: Apple Find My & Privacy page and privacy pages; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real system's design, numbers or controls.

Follow-ups.

- **Q.** How do you let a family member legitimately see a device's location? **A.** "Through explicit, revocable sharing: the owner invites the person, the shared person's device shows that sharing is on, and either party can end it at any time with a notification. Sharing is by account permission, never by exposing the underlying signal, and it never extends to third parties."
- **Q.** What if the owner is the abuser, and the "lost" device belongs to the victim? **A.** "The design assumes this can happen. The affected person must be able to find unknown trackers near them, see and remove devices linked to their accounts, and get guidance from support, including help for people at risk. The system cannot tell an abusive owner from a legitimate one by looking at location, so it relies on alerts to the person being tracked and on making covert use difficult. I would involve safety experts and advocacy groups in review, and accept that no design removes the risk entirely."

IQ-C22-07 · Hard · system design · SD · Amazon

Prompt. Design order tracking and delivery-status updates for an online store.

Rubric.

- **Baseline:** "Show the order status from the database."
- **Strong:** Handles events from carriers, notifications, and load at peak.

- **Standout:** Handles duplicate and out-of-order events, a state machine, eventual consistency, graceful degradation, and partner variability.

Model answer. The setting is general; nothing here describes any real design. The job: a customer sees where their order is and gets updates, even when carriers report late, twice or out of order.

Requirements. Show a status and a timeline; push key updates (shipped, out for delivery, delivered, delayed); handle many carriers with different capabilities. Freshness: updates within a minute of a carrier event. Must never: tell a customer an order is delivered when it is not, or send the same alert repeatedly.

Scale (invented). 20 million orders a day with about 8 tracking events each: 160 million events a day, about 1,850 a second, and about 11,100 at a peak factor of 6 (a holiday). Customers check tracking about 3 times per order: 60 million views a day, about 700 a second, with spikes on delivery days.

Sketch. Inbound: carriers send updates by webhook, or the system polls carriers that cannot push. Each event goes to an intake service that verifies the source, assigns an identifier, and puts it on a queue. A processor applies the event to the order's tracking record. *State machine:* an order moves through states (label created, in transit, out for delivery, delivered, exception); the processor accepts only valid transitions, so a stray "in transit" after "delivered" does not move the order backward, and ignores events with an already-seen identifier. *Outbound:* the status page reads the tracking record (cached for seconds); notifications go through the notification pipeline (C22 Worked example 1) with preferences.

Duplicates and ordering. Carriers repeat events and send them out of order. The processor deduplicates by event identifier and orders by the carrier's event time, not by arrival time, within a tolerance. Eventual consistency is fine: seconds of lag are harmless, but the status must converge to the true state.

Failure and degradation. If a carrier's feed is down, show the last known status with its timestamp ("last updated 3 hours ago") and the expected delivery window, not a blank; alert the carrier-integration team. If the queue backs up at peak, process delivered and exception events first. The page must degrade to a static status when the timeline service fails.

Trust risks. A false "delivered" (a driver scans early) causes complaints; add signals (location, photo proof) where available, and a one-click "not received" path.

Measures. Event-to-display delay (95th percentile under 60 seconds), share of orders with tracking updates, "where is my order" contacts per 1,000 orders, false-delivered reports.

Company note. Amazon is the setting only: Inferred from candidate reports of order and fulfillment design prompts [unverified: candidate-reported] . No Amazon data or design is claimed.

Follow-ups.

- **Q.** A carrier sends "delivered" and then, a minute later, "out for delivery." What do you do? **A.** "Treat the second event as out of order or a mistake: keep the order in 'delivered' because the state machine does not go backward without an explicit exception, but

record the odd event, and flag the order for review if the customer reports non-receipt. The timeline can show both with timestamps."

- **Q.** How do you handle a carrier with no electronic updates? **A.** "Poll its public tracking where permitted, use scans at our own facilities, accept less frequent updates and say so ('updates may be delayed'), and push the carrier toward a feed in the contract. The customer's view is honest about the data's freshness."

IQ-C22-08 · Hard · system design · SD · Netflix

Prompt. Design how a streaming service starts playing a video quickly and keeps playing smoothly.

Rubric.

- **Baseline:** "Stream the video from a server."
- **Strong:** Names the content network, adaptive quality and buffering.
- **Standout:** Computes startup time under different networks, explains starting low and ramping, covers failover and regional capacity, and sets measures.

Model answer. The setting is general; nothing here describes any real design. The job: press play and see video within about two seconds, without stalls, on any connection.

Requirements. Time to first frame under about 2 seconds at the 95th percentile; very few stalls; quality as high as the connection allows; works on slow networks and old devices. Must not: show an endless spinner.

Sketch. Video is stored as many short segments (about 2 seconds each) at several quality levels. *Start:* the app asks the service to authorize playback and gets a manifest (a list of segments and quality levels); it fetches segments from a nearby server in a content delivery network, which holds popular titles close to viewers. *Adaptive quality:* the player measures its download speed and chooses the quality of each next segment, moving up when it can and down when it must.

The startup arithmetic. A 2-second segment at 3 Mbps is 0.75 MB. On a 10 Mbps connection it downloads in 0.6 seconds; on a 2 Mbps connection, in 3.0 seconds. If instead the player starts at a lower quality of 0.75 Mbps, the first segment is about 0.19 MB and downloads in 0.75 seconds on the 2 Mbps connection. So starting low and ramping up cuts startup on slow connections from about 3 seconds to under 1, at the cost of a few seconds of lower picture quality, which most viewers prefer to waiting.

Capacity and placement. Popular titles are copied to servers near viewers ahead of demand, predicted from what viewers watch; less popular titles are fetched on demand. Before a major release, copy the title to regions in advance and reserve capacity; stage availability by region if demand is uncertain (C22 CS-C22-02).

Failure and degradation. If the nearest server fails or is overloaded, the player switches to another source mid-stream; if bandwidth drops, the player lowers quality rather than stall; if the authorization service is slow, a short-lived cached permission lets recent viewers start. The viewer should see at worst a reduced picture, not a stall.

Measures. Time to first frame (median and 95th percentile), failed starts, rebuffering time per hour, average quality delivered, with guardrails on cost per hour streamed.

Company note. Netflix is the setting only: **Public** that Netflix has published on its content delivery network and on resilience testing, **Inferred** from candidate reports of playback and graceful-degradation prompts [V: Netflix Tech Blog on Open Connect and resilience testing; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real system's numbers or design.

Follow-ups.

- **Q.** How do you prepare for a huge release, say a season everyone has waited for? **A.** "Predict demand by region, pre-position the files near viewers before release, reserve network and server capacity, lower the starting quality briefly if needed, and watch the stall and failed-start rates live with a plan to shed quality, not viewers. A staged rollout by region helps if the forecast is uncertain."
- **Q.** Why not always start at the highest quality? **A.** "The first segment at high quality is large; on a slow connection that is the long wait the viewer notices. Starting low and ramping up delivers a fast start and reaches high quality within seconds on a good connection."

IQ-C22-09 · Hard · system design · SD · Stripe

Prompt. Design webhook delivery for a payments platform so that merchants never miss an event.

Rubric.

- **Baseline:** "Send an HTTP request to the merchant when something happens."
- **Strong:** Covers retries, signatures, and duplicates.
- **Standout:** Explains at-least-once delivery, backoff and its load effects, ordering, replay and catch-up, merchant-side idempotency, and monitoring.

Model answer. The setting is general; nothing here describes any real system. The job: tell a merchant's system that something happened (a payment succeeded, a refund was issued) reliably, even when the merchant's server is slow or down.

Requirements. Every event is delivered at least once; most within seconds; no forged events; merchants can recover missed events; the platform is not overwhelmed by a merchant's outage. Non-goals: guaranteed ordering.

Sketch. When an event occurs, the platform records it in an event log with a unique identifier, then enqueues a delivery task. A delivery worker sends the event to the merchant's registered address, signed so the merchant can verify its origin, and expects a success response within a short time (say 10 seconds). On failure or timeout, the task is retried later according to a schedule.

Retry schedule and the arithmetic. Suppose 100 million events a day: about 1,160 a second. If 2% fail on the first attempt, that is 2 million retries needing a second attempt. A growing schedule (for example 1 minute, 5 minutes, 30 minutes, 2 hours, 12 hours, 24 hours: six

attempts over a day) spreads the retry load. If a merchant's endpoint is down for an hour while receiving 1,160 events a second of platform traffic for it (a large merchant), about 4.2 million events accumulate; without backoff the platform would hammer a server that is already down, and with backoff and a per-merchant limit it does not.

Duplicates and order. Because delivery is at least once, a merchant can receive the same event twice; each event has an identifier, and merchants should record processed identifiers and ignore repeats. Events can arrive out of order; the platform tells merchants to treat the event as a notification and fetch the current state from the API if order matters.

Replay and catch-up. After a long outage, a merchant needs the events it missed: provide an API to list events from a given time and a dashboard button to resend. Mark endpoints that fail for days as disabled with an alert to the merchant.

Security. Sign each delivery with a secret and include a timestamp so that old messages cannot be replayed by an attacker; deliver only over secure connections.

Failure and monitoring. Merchants see delivery status and errors per event; the platform alerts on rising failure rates and on the oldest undelivered event. A slow endpoint (not down) is treated as a failure after the timeout and subject to a concurrency limit, so one slow merchant cannot tie up the workers.

Measures. Share of events delivered on the first attempt, time to delivery (95th percentile), events outstanding after 24 hours, merchant-reported missed events (target zero).

Company note. Stripe is the setting only: `Public` that Stripe's documentation describes webhooks with retries and signatures, `Inferred` from candidate reports of developer-delivery design prompts [V: Stripe API documentation; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real system's internals.

Follow-ups.

- **Q.** What if a merchant's endpoint is slow but not down? **A.** "Treat a response slower than the timeout as a failure and retry later, and cap how many deliveries are in flight to that endpoint so it cannot occupy all workers. Tell the merchant through the dashboard that its endpoint is slow, with the typical response time, so they can fix it."
- **Q.** How can a merchant catch up after a three-day outage? **A.** "The event log is the source of truth: the merchant lists events since the outage began through the API and processes them, skipping identifiers it has seen. Retries stop after the schedule ends, so replay is the safety net."

IQ-C22-10 · Intermediate · system design · SD

Prompt. Explain the design of your notification system to the CEO in three minutes.

Rubric.

- **Baseline:** A technical walk through the components.
- **Strong:** Leads with the user outcome and the main risk.
- **Standout:** States the decision needed, cost and time in plain terms, explains the key tradeoff and the biggest risk with a mitigation, and offers a next step.

Model answer. I'd lead with what matters to them and drop the boxes.

"The outcome. People get the right notification at the right time, once, on the channel they prefer, which brings them back without annoying them. Today we send some twice and some late.

How it works, in one picture. Events from the product go into one pipeline that checks each person's preferences, removes duplicates, groups bursts, and sends by push or email through our providers. If a provider is slow, messages wait and retry; important ones, like security alerts, go first.

The numbers. About 160 million notifications a day, with evening peaks about five times the average, around 9,000 a second. Our push provider's limits are the thing to watch at that level.

The main tradeoff. We can send instantly, or group messages for a few seconds to cut annoyance and cost. I recommend short grouping for messages and longer for likes.

The biggest risk. Spam and abuse: bots mass-mentioning people. We cap how many a person can receive from one sender and give users controls to mute and report.

Cost and time. The first version takes one team about a quarter; running cost is mostly provider fees, roughly proportional to volume, and grouping lowers it.

What I need from you. A decision to launch to 1% of users next month with guardrails on opt-outs and uninstalls, and agreement that security alerts get priority over everything else. I'll report weekly."

Follow-ups.

- **Q.** The CEO asks, "Can we launch next month?" **A.** "A limited launch to 1% of users, yes. Full launch depends on two things we will know in four weeks: whether the provider's limits hold at our peak and whether opt-out rates stay flat. I will tell you the result on a date, and I would not promise the full launch before then."
- **Q.** The CEO asks, "Why does this cost so much?" **A.** "The cost is mostly sending: every message to a phone or inbox has a fee, and we send 160 million a day. Grouping bursts and preferring push over email are the two levers; I can show you the cost per user per month and what each lever saves."

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C22-01 · Estimate a chat app

Problem. A chat app has 30 million daily users, each sending 40 messages a day, at 0.5 KB a message. Estimate the average and peak writes per second (peak factor 5), the daily storage and the storage for a year, and say what each number suggests.

Context. A first estimate.

Data. As above.

Constraints. Five minutes.

Expected approach. FW-C22-03.

Solution. Messages a day: $30,000,000 \times 40 = 1.2$ billion; per second: $1,200,000,000 \div 86,400 =$ about 13,900; peak: about 69,400 a second. Storage a day: 1.2 billion $\times 0.5$ KB = 600 GB; a year: about 219 TB. Implications: 69,400 writes a second at peak is beyond a single database, so the data must be spread across machines (ask engineering about partitioning by conversation); 600 GB a day is modest, so the cost question is retention and replication (three copies triple it); and old messages can move to cheaper storage.

Alternative solution. Add the read side: if each message is read by 3 recipients on average, reads are about 3 times writes.

Tradeoffs. Keeping all messages forever has a storage cost and a privacy risk; ask what retention users expect.

EX-C22-02 · A requirements card

Problem. Fill in a requirements card (FW-C22-02) for an event-ticketing system for concerts, where a popular concert's tickets sell out in minutes.

Context. The first five minutes of a design round.

Data. None.

Constraints. Six minutes.

Expected approach. FW-C22-02.

Solution. *Users and job:* fans buying tickets; promoters selling them. *Must do:* show availability, hold tickets during checkout for a few minutes, take payment, issue a ticket. *Must not:* sell the same seat twice; let bots buy everything. *Latency and freshness:* availability within seconds; checkout steps under 2 seconds. *Availability and durability:* the sale must stay up during the spike; a paid ticket is never lost.

Scale: normal traffic is small; at on-sale, perhaps a million people arrive within five minutes (a very high peak factor). *Privacy and constraints:* payment data; resale rules by region. *Non-goals:* resale marketplace in version 1. *Success measure and guardrail:* tickets sold to real fans without oversell; guardrails: checkout abandonment, bot share.

Alternative solution. Add a virtual waiting room as a requirement, since it shapes the whole design.

Tradeoffs. A waiting room protects the system and turns some people away; say how you decide who goes first.

EX-C22-03 · The spike factor

Problem. A ticket on-sale normally sees 50 requests a second. At the on-sale, 1,000,000 people arrive within 5 minutes, each making one request to the sale page. Compute the request rate and the ratio to normal, and say what design response it suggests.

Context. Spike design.

Data. As above.

Constraints. Five minutes.

Expected approach. C22.5 and CS-C22-02.

Solution. $1,000,000 \div 300 \text{ seconds} = \text{about } 3,300 \text{ requests a second}$, about 67 times the normal 50. A system sized for the average cannot absorb it, and sizing for $67\times$ all year is wasteful. Responses: a waiting room that admits people at a rate the system can serve (for example 500 a second), static content served from a content network, elastic capacity added before the on-sale, and load shedding for non-essential features during the peak. If people are admitted at 500 a second, a million people take 2,000 seconds (33 minutes), which is a product decision about fairness and communication.

Alternative solution. Randomize the order of those who arrive in the first minutes so that speed of clicking does not decide.

Tradeoffs. A waiting room trades system stability for some users' waiting.

EX-C22-04 · Where to stop fanning out

Problem. A feed system can write posts into followers' feeds at 20,000 writes a second for this job, and a post should reach all followers within 60 seconds. What is the largest follower count for which writing to every follower meets the target? What do you do above it?

Context. The celebrity problem.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 2.

Solution. In 60 seconds at 20,000 writes a second the job can write $20,000 \times 60 = 1,200,000$ feeds. So accounts up to about 1.2 million followers meet the target, with no margin; set the threshold lower (for example 500,000) to leave room for other posts. Above it, do not write to every follower: store the post once and merge it into followers' feeds at read time. The cost is a few extra reads for users who follow large accounts.

Alternative solution. For very large accounts, write to the feeds of the most active followers first and let the rest read at open.

Tradeoffs. Hybrids are more complex; they bound the worst case.

EX-C22-05 · Push or poll

Problem. An app shows live order status to 5,000,000 active users. Compare polling every 30 seconds with pushing updates. Assume each user's order changes status about 4 times an hour and 3% of polls return new data. Compute the request rates and the wasted share.

Context. A design choice.

Data. As above.

Constraints. Six minutes.

Expected approach. C22.6.

Solution. Polling: $5,000,000 \div 30 =$ about 166,700 requests a second, of which 97% (about 161,700 a second) return nothing new. Pushing: $5,000,000 \times 4 \div 3,600 =$ about 5,560 messages a second, each carrying a real update. Push uses about one thirtieth of the traffic and delivers updates immediately, but it requires holding connections or using the phone's push service, and handling devices that are offline. A mixed design: push when available, poll slowly (every few minutes) as a fallback.

Alternative solution. Poll adaptively: fast while the order is out for delivery, slow otherwise.

Tradeoffs. Push reduces waste and adds connection management and delivery guarantees.

EX-C22-06 · Failure-mode table for a ride request

Problem. Build a failure-mode table (FW-C22-06) for a ride-hailing request: components are the app, the dispatch service, the location store, the driver app and the payment service. Give one failure each, what the rider sees and the degradation.

Context. Resilience design.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C22-06.

Solution. App: loses connection after requesting; rider sees "reconnecting" and the request status is recovered on reconnect (the request is stored server-side). Dispatch service: slow; rider sees "finding a driver" with a longer estimate; degrade by widening the search radius and queueing requests. Location store: stale driver positions; rider sees cars that are not there; degrade by showing a coarse estimate instead of precise car icons and labeling the age of data. Driver app: driver loses connection; rider sees the trip continuing; the system keeps the last known position and prompts the driver to reconnect; after a timeout the trip is flagged. Payment service: unavailable; rider completes the ride and the charge is deferred and retried, with the fare shown as pending, rather than blocking the ride.

Alternative solution. Add a signal for each (the metric that would alert you).

Tradeoffs. Deferring payment accepts some uncollectible fares in exchange for riders not being stranded.

EX-C22-07 · An abuse-case pass for live location sharing

Problem. Do an abuse-case pass (FW-C22-08) for a feature that lets a user share their live location with friends for an hour.

Context. Safety review.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C22-08.

Solution. *A stalker or abusive partner* pressures someone into sharing, or shares a phone with sharing preset: harm is coercive tracking; controls are a visible indicator while sharing, a notification when sharing starts, a time limit that cannot be extended silently, one-tap stop, and safety guidance. *A stranger* tricks a user into sharing through a fake link: controls are sharing only with existing contacts, a confirmation that names the recipient, and a warning for new contacts. *A friend who leaks* the location to others: controls are no forwarding of the live view, and a note that the recipient can see it. *A minor sharing with an adult stranger:* controls are age-based defaults, no sharing with non-contacts, and parental settings where appropriate. Residual risk: coercion by someone with physical access to the phone cannot be eliminated; accept it, document it and provide support resources.

Alternative solution. Add a "safety check" prompt after an hour asking whether the user wants to continue.

Tradeoffs. Each control adds friction for honest users; keep friction proportional to harm.

EX-C22-08 · Consistency choices

Problem. For each data item choose strong or eventual consistency and say why: (a) a user's account balance; (b) the number of likes on a post; (c) the available seats on a flight at checkout; (d) a user's profile photo; (e) a delivery driver's current position.

Context. Design tradeoffs.

Data. None.

Constraints. Five minutes.

Expected approach. FW-C22-05.

Solution. (a) Strong: a wrong balance can cause overdrafts or double spending. (b) Eventual: a few seconds of lag is harmless. (c) Strong at the moment of booking: two people must not get the same seat; the search list may be eventually consistent but the reservation step checks the source of truth. (d) Eventual for other viewers, but the owner should see their own change at once (read-your-own-writes, C21). (e) Eventual with a freshness label: the position is old within seconds anyway, and showing its age is honest.

Alternative solution. For (c), allow brief overselling and compensate (rebooking), if the business accepts that cost, as some airlines do.

Tradeoffs. Strong consistency costs speed and availability; reserve it for where mistakes are expensive.

EX-C22-09 · Find the bottleneck

Problem. A system's parts handle these loads: load balancer 50,000 requests a second, application service 8,000 requests a second, queue 20,000 messages a second, database writes 3,000 a second. The expected peak is 6,000 requests a second, each causing one database write. Where is the bottleneck, by how much, and what are the options?

Context. Capacity review.

Data. As above.

Constraints. Six minutes.

Expected approach. C22.5 and Amdahl's law (C21, mental models, section 9).

Solution. The database writes (3,000 a second) are the tightest limit: the peak needs 6,000, twice its capacity. The application service (8,000) has 33% headroom over the 6,000 peak, and the load balancer and queue are far above. Options: put the writes on the queue and let a worker write at the database's pace (the user sees "processing," and the backlog drains after the peak); batch writes; split the data across two databases so that each handles 3,000; or reduce writes per request. The queue option is cheapest if a short delay is acceptable; splitting the data fixes capacity without a delay and costs more engineering.

Alternative solution. Cache and combine updates to the same record so fewer writes reach the database.

Tradeoffs. Asynchronous writes need idempotency and a status the user can check.

EX-C22-10 · A timed full design

Problem. Give a 30-minute answer to: "Design the system that shows a food-delivery customer the courier's live location and an estimated arrival time."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Label what you assume.

Constraints. Use the 45-minute budget in C22.2, compressed.

Expected approach. The spine (FW-C22-01), scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Clarify: the customer sees a map and an estimate, updated every few seconds, from order pickup to delivery; privacy (location shown only to the customer for that order, only during the delivery); battery on the courier's phone. Estimate (invented): 2 million deliveries a day, with 30 minutes of tracking each at one update per 5 seconds: 360 updates per delivery, 720 million updates a day, about 8,300 a second, and about 25,000 at a peak factor of 3; each update is small (about 100 bytes), so about 72 GB a day if kept, and only the latest position is needed live. Sketch: the courier app sends positions to a location service that keeps the latest in a fast store; the customer app receives updates by push (or polling every few seconds as a fallback); an estimate service

combines position, route and traffic. Tradeoffs: update frequency against battery and cost; precision against privacy (fuzz the position near the customer's home until arrival); freshness against cost.

Failure: if the courier loses signal, show the last position with its age and widen the estimate; if the estimate service fails, show the last estimate and label it. Abuse: only the customer of that order can see the courier, and for a limited time; the courier's privacy after delivery. Measures: 95th-percentile age of the displayed position, estimate error, battery impact, "where is my courier" contacts. Rollout: one city first.

Alternative solution. Show the estimate and a coarse progress bar and omit the live map, to reduce cost and privacy exposure.

Tradeoffs. A live map delights customers and costs battery, data and privacy; a progress bar is cheap and less satisfying.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: requirements and estimates.* Use IQ-C22-01, IQ-C22-02, EX-C22-01, EX-C22-02 and EX-C22-03. Fill a requirements card and an estimate sheet for a new prompt each day, saying the decision each number drives.
- *Week 2: designs and tradeoffs.* Use IQ-C22-03, IQ-C22-04, IQ-C22-05, IQ-C22-07, EX-C22-04, EX-C22-05 and EX-C22-08. Draw the write and read paths from memory and defend one tradeoff aloud.
- *Week 3: failure, abuse and communication.* Use IQ-C22-06, IQ-C22-08, IQ-C22-09, IQ-C22-10, EX-C22-06, EX-C22-07, EX-C22-09 and EX-C22-10. Run a full 30-minute design with a partner who changes the scale midway and asks "who could abuse this?"

With a partner. Ask your partner to interrupt with "Why that?" and "What if it fails?" at random. Practice answering in one sentence tied to the user, then returning to your plan.

Read a real design. Read a public engineering write-up of a system you use (a notification, feed or payments design) and extract its requirements, numbers and tradeoffs in one page.

Feedback checklist (yes, partly, no).

1. Did I clarify users, job, scale and constraints before drawing?
2. Did I estimate average and peak load and storage, and name the decisions they drive?
3. Did I sketch the write and read paths and mark where the user waits?
4. Did I name the tradeoff and choose with a user reason?
5. Did I say what the user sees when each part fails?
6. Did I run an abuse-case pass where people or money are involved?
7. Did I choose consistency and delivery guarantees by the cost of a wrong result?
8. Did I avoid naming technology without a reason?

9. Did I close with an executive summary and a question for engineering?
10. Did I stay within the time?

Answer journal. For each drill record the date, the prompt, the numbers you used, the main tradeoff, the failure and abuse cases, your score out of ten and the step you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. Each deliverable doubles as an interview story.

Days 1 to 3. Pick a feature your team owns and write its requirements card (FW-C22-02), including the non-functional requirements nobody wrote down (freshness, durability, consistency, privacy). Review it with the engineering lead and note what was missing.

Week 1. Estimate its load with the sheet (FW-C22-03): average, peak, storage, and one cost figure. Compare with what engineering knows; note the biggest discrepancy and why.

Week 1. Draw the write and read paths for one core action with an engineer, marking what the user waits for and what is asynchronous.

Week 2. Build the failure-mode table for the feature (FW-C22-06): for each component, what the user sees when it fails. Take one gap (a blank screen, a confusing error) to design.

Week 2. If the feature touches people, location, messages or money, run an abuse-case pass (FW-C22-08) with security or trust and safety. Record the controls and the residual risk.

Week 3. Write the design one-pager (FW-C22-07) in an engineer version and an executive version, and test each on someone from that audience.

Deliverables that become stories: the requirements card with the missing requirements (a time you framed a problem so a team could design well), the failure and abuse tables (a time you anticipated what could go wrong), and the two-audience one-pager (a time you communicated a technical design). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. A PM-level design answer is a product decision expressed as a system: start with users, the core job, scale and constraints, state assumptions, and let numbers drive choices, not technology names.
2. Estimate with a stated method (daily users $\times$ actions $\div$ 86,400, a peak factor, item sizes and retention), say what decision each number drives, and sanity-check; peaks, not averages, size the system.

3. Sketch a handful of components and walk a write and a read through them, marking what the user waits for; choose consistency, freshness and delivery guarantees by the cost of a wrong, stale or duplicated result.
4. Name the tradeoff dials (latency against freshness, consistency against availability, cost, privacy against capability, push against pull) and choose with a reason tied to the user.
5. Design for failure and misuse: say what the user sees when each part fails and how it degrades, run an abuse-case pass wherever people, location, messages or money are involved, and close with a one-page summary an engineer can act on and an executive can follow.

Further reading

- Martin Kleppmann, *Designing Data-Intensive Applications* (2017), for the ideas behind consistency, queues and replication [V: O'Reilly, March 2017, ISBN 9781449373320].
- Betsy Beyer and others (eds.), *Site Reliability Engineering* (2016), on capacity, overload and graceful degradation [V: Site Reliability Engineering, O'Reilly, 2016].
- Alex Xu, *System Design Interview: An Insider's Guide*, Volume 1 (2020), as an example of how engineering interviews treat these designs; read it for the vocabulary and apply the PM-level bar from this chapter [V: Volume 1, 12 June 2020, ISBN 9798664653403 (independently published; Byte Code)].
- Jeff Dean's widely circulated list of latency numbers for computers (slides "Numbers Everyone Should Know" from his talk on building large-scale distributed systems; popularized as "Latency Numbers Every Programmer Should Know"; the figures are about a decade old and later estimates differ) [V: Dean's talk slides; later estimates differ], and Little's law in any standard queueing reference [as of 2026-10].
- Luke Stone, "Bringing Pokémon GO to life on Google Cloud," Google Cloud blog, 29 September 2016 (cloud.google.com/blog/products/containers-kubernetes/bringing-pokemon-go-to-life-on-google-cloud). The source for CS-C22-02.
- Public engineering write-ups of notification, feed and payments systems from large technology companies, as worked examples.